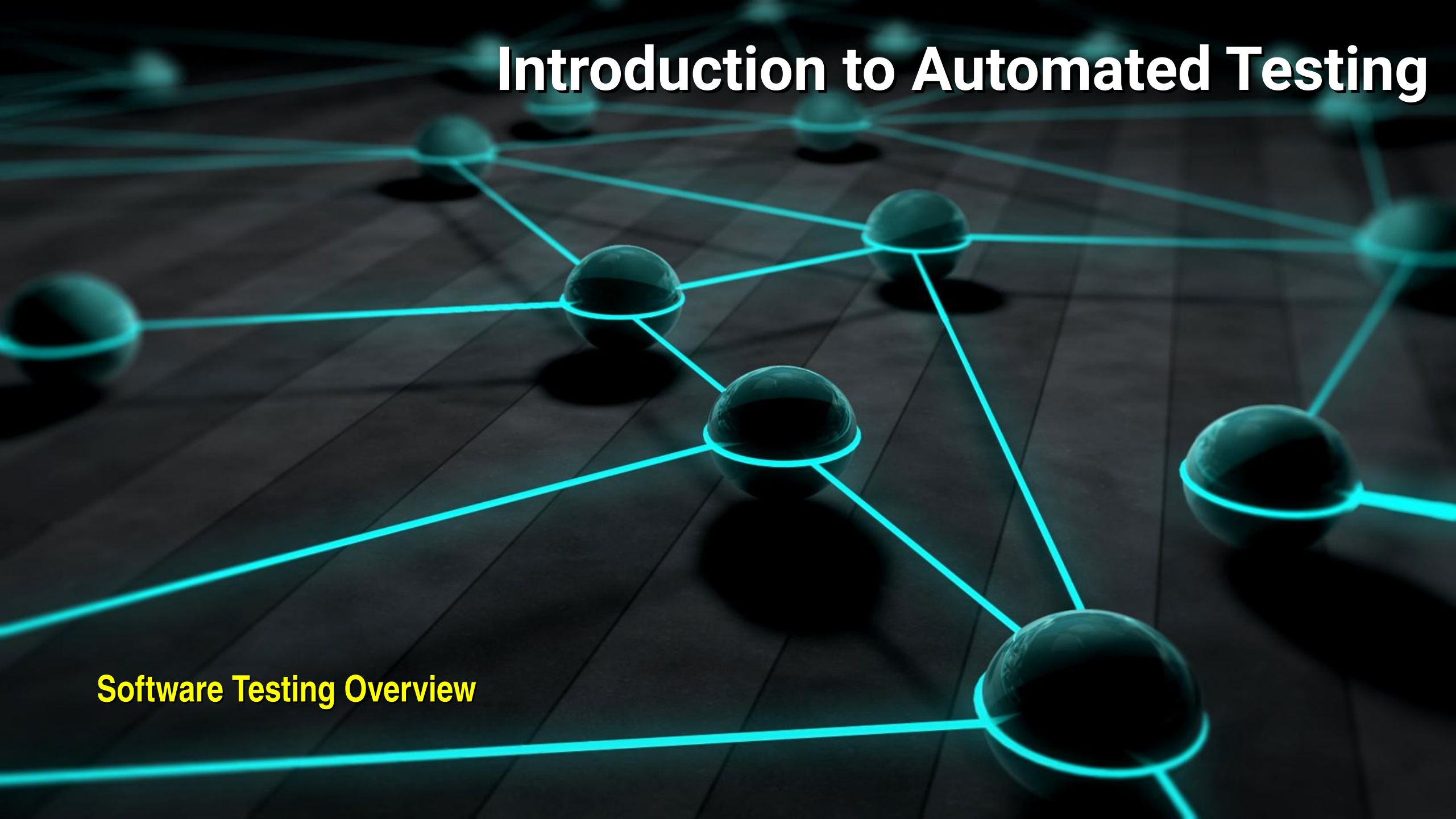


Introduction to Automated Testing

The background of the slide features a network diagram. It consists of several glowing blue spheres of varying sizes, connected by thin, bright blue lines. These nodes and connections are arranged in a complex, web-like pattern across the frame. The spheres have a reflective, metallic appearance. The entire scene is set against a dark, almost black background that has a subtle grid or wood-grain texture. The lighting is focused on the glowing nodes, creating a sense of depth and connectivity.

Software Testing Overview

The Unit Testing Strategy

- Complexity
 - Consider a machine that has a single steel part
 - This machine can fail in one way – the part can break
 - Add a second part the same as the first part, and bolt the two together
 - Now the machine can fail in 7 ways – if we blindly add a test case for each possible combination, the number of test cases increases exponentially
 - The unit testing strategy is to break the complex machine into three components and then perform three unit tests
 - That provides us with enough information to determine if each unit tests pass, then we do one integration test and we

Part	1	2	3	4	5	6	7	8
Part 1	P	P	P	P	F	F	F	F
Bolt	P	P	F	F	P	P	F	F
Part 2	P	F	P	F	P	F	P	F



The Unit Testing Strategy

- Basic approach
 - Select the component to be unit tested, it can be any size from a piece of code to a subsystem
 - Create a driver
 - This is stand-in for the rest of the system
 - It feeds the test cases as input to the component and records the component's response
 - Any test failures are due to the component since we have sanitized the inputs
 - Create stubs (mocks)
 - Anytime the component needs to call another component
 - A stub is designed to return exactly the right response to each request the component makes
 - We know the correct replies in advance because they are determined by the test cases
 - For example, a database stub just has a list of the right responses for the queries that will be sent when the component executes the test cases
 - The test failures now are isolated to the component under test
 - We are testing only one thing (unit = one) which is the component under test



Testing Methods

- Returning to the concept of test efficiency
- There are a number of ways to reduce the amount of testing without significantly impacting the effectiveness of the testing
- This is just a high level introduction
- Equivalence classes
 - Method for grouping test cases by identifying various classes of input conditions with the assumption that each member of a class causes the same kind of processing and output to occur as any other member of the class
 - Assumes that inputs that are subject to the same logic in the spec are processed by the same code – assumes that the code is competently written



Testing Methods

- A group of tests forms an equivalence class if
 - They all test the same thing
 - If one test case in an equivalence class detects a defect, then every other test case in the same equivalence class will detect the same defect
 - If one test case in an equivalence class fails to detect a defect, then every other test case in the same equivalence class will fail to detect the same defect
- Reduces an infinite number of inputs to a finite number of classes
 - The we write a test using a representative data point from that class
 - Turns out to be very effective in real life when coupled with other methods



Testing Methods

- Boundary Value Analysis
 - The boundaries between equivalence classes is where many defects occur
 - Usually decision logic errors
 - For example, does “up to four” mean up to and including four or up to but not including four?
 - Can be applied to any data that can be ordered
 - Test cases are chosen by taking
 - The largest valid value in the class and the smallest valid value in the class
 - The smallest value that is too large
 - The largest value that is too small
 - Other values around the boundary as necessary

AGE *Accepts value 18 to 56

BOUNDARY VALUE ANALYSIS		
Invalid (min -1)	Valid (min, +min, -max, max)	Invalid (max +1)
17	18, 19, 55, 56	57



Generating Unit Tests

- Unit tests are not generated in isolation
- They are derived from the acceptance tests
 - The acceptance tests and system architecture determine the inputs for a component
 - The acceptance tests that would result in an call to the component are isolated
 - The input each test provides is associated with what the correct output should be
 - This usually reduces the number of inputs
- The unit tests remain stable unless there is change in the architecture or the acceptance tests for the system



Generating Unit Tests

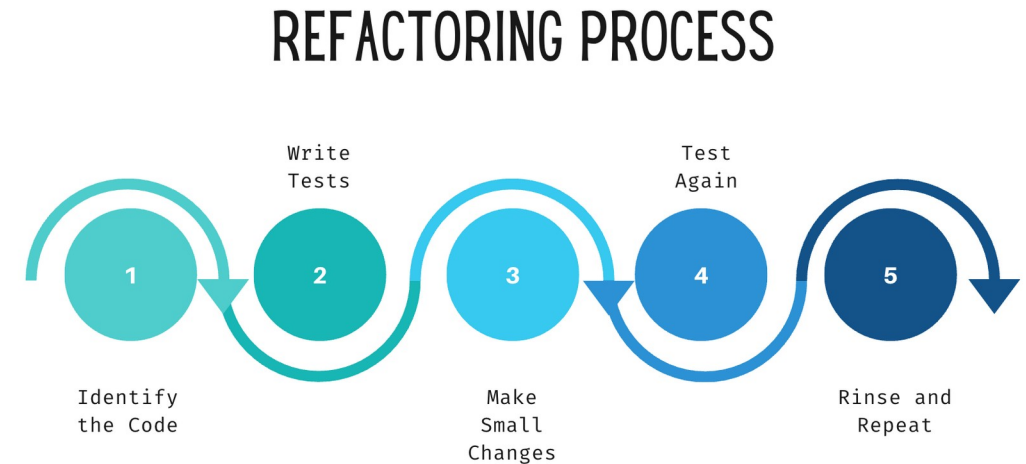
- IEEE Best Practices expressed in terms of acceptance tests

Complete	<i>The acceptance tests cover all possible inputs and conditions, both valid and expected as well as invalid and unexpected.</i>
Consistent	<i>No two acceptance tests require to system to behave differently when the state and input of two tests are the same.</i>
Correct	<i>The expected result of each test meets the acceptance criteria.</i>
Testable	<i>All test inputs, states and outputs are quantified and measurable.</i>
Verifiable	<i>There is a finite cost effective process for executing each test.</i>
Unambiguous	<i>There is only possible way to interpret or understand each test and test result.</i>
Valid	<i>Everyone can read, understand, and analyze the tests well enough to formally approve the tests.</i>
Modifiable	<i>The test cases are organized in a way so that they are easy to use, modify and update.</i>
Ranked	<i>The test cases are in a priority order that everyone on both the business and technical sides agree on.</i>
Traceable	<i>Every test case can be traced back to the example and acceptance criteria that motivated it and then back to the original requirement.</i>



Refactoring

- Tests as scaffolding for refactoring
 - Tests let you change code with confidence
 - Without tests, every change is a gamble
 - Did I break something?
 - With tests, refactoring is safe
 - The tests catch you when you slip
 - This is often the biggest day-to-day value of a test suite



Refactoring

- Tests for refactoring
 - Need to be automated
 - Otherwise development is impeded
 - Too much development time is spent on writing and running tests
 - Tests ensure functional correctness
 - While code structure is optimized
 - Ensures that technical debt is paid off immediately
 - Reduces downstream maintenance load



Inherited Code

- Much of the legacy code you'll test was written without tests
 - Organizations often have decades of code written before automated testing was standard
 - Adding tests to legacy code is harder than writing tests with the code
 - The strategy is to add tests when you change code, not in a heroic top-down sweep
 - Imperfect coverage that grows is better than perfect coverage that doesn't exist

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i < m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram shows a Java code snippet for the `addAll` method of an `AbstractCollection` class. The code uses a series of `if-else if` statements to handle different types of `AbstractCollection` objects: `Set`, `List`, and `Map`. Annotations with colored lines point to specific parts of the code:

- Duplicated Code** (blue text): Points to the `if (c instanceof Set)` block and the `if (c instanceof List)` block, indicating that the logic for adding elements is repeated.
- Duplicated Code** (purple text): Points to the `if (!contains(s.getElementAt(i)))` and `if (!contains(l.get(i)))` conditions, indicating duplicated logic.
- Alternative Classes with Different Interfaces** (green text): Points to the `Set` and `List` types, suggesting that these classes might have different interfaces or methods.
- Switch Statement** (red text): Points to the `if-else if` structure, suggesting that a `switch` statement might be a more appropriate way to handle the different types.
- Inappropriate Intimacy** (red text): Points to the `m.keys[i]` and `m.values[i]` access, suggesting that this is an inappropriate level of access to the `Map` object.
- Long Method** (black text): Points to the entire `addAll` method, indicating that the method is too long and should be refactored.



Inherited Code

- When to write a test for legacy code
 - When you fix a bug
 - Write a test that would have caught it
 - Coverage goes up by one test, and you have a regression test for the bug
 - When you add a feature
 - Write tests for the new code
 - The new code is testable because you wrote it that way
 - The old code remains as it was.

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i < m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram highlights several code quality issues in the `addAll` method:

- Duplicated Code**: Points to the `if (!contains(s.getElementAt(i)))` and `add(s.getElementAt(i));` block in the `Set` branch, and the `if (!contains(l.get(i)))` and `add(l.get(i));` block in the `List` branch.
- Alternative Classes with Different Interfaces**: Points to the `if (c instanceof Set)` and `else if (c instanceof List)` branches.
- Switch Statement**: Points to the `else if (c instanceof Map)` branch.
- Inappropriate Intimacy**: Points to the `m.keys[i]` and `m.values[i]` access in the `Map` branch.
- Long Method**: Points to the entire `addAll` method.



Inherited Code

- When you refactor a hard-to-test class
 - Take small steps
 - Extract a method
 - Give it dependencies it can be supplied with
 - Write a test for the extracted piece
 - The class as a whole is still hard to test
 - One piece of it now is testable.

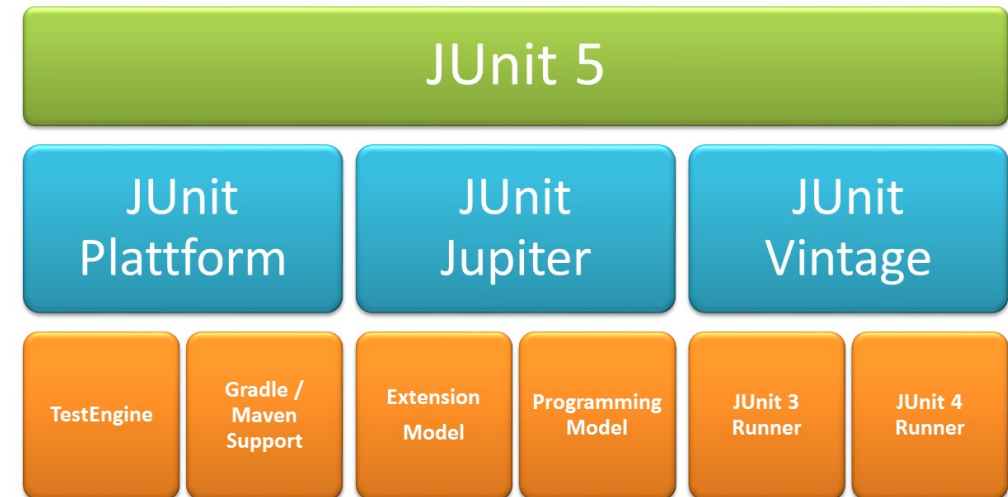
```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i < m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram highlights several code quality issues in the `addAll` method:

- Duplicated Code**: Points to the `if` statement and the `for` loop in the `Set` branch.
- Duplicated Code**: Points to the `if` statement and the `for` loop in the `List` branch.
- Alternative Classes with Different Interfaces**: Points to the `Set` and `List` branches, indicating that different collection types are handled differently.
- Switch Statement**: Points to the `if-else if` structure, suggesting a switch statement might be more appropriate.
- Inappropriate Intimacy**: Points to the `m.keys[i]` and `m.values[i]` access in the `Map` branch, indicating that the method is accessing internal details of the `Map` interface.
- Long Method**: Points to the entire `addAll` method, indicating that it is too long and should be refactored.

JUnit 5 and JUnit 4

- What's different from JUnit 4
 - Modular
 - Separate engine, API, parameter resolution
 - Java 8 baseline
 - Lambdas in assertions, streams in test data
 - Native parameterized tests
 - No separate runner needed
 - Better extension model
 - @ExtendWith for things like Mockito and Spring
 - Default visibility
 - Tests can be package-private (no need for public)



Test Method

- Elements of a test method
 - @Test marks the method as a test
 - Method body follows AAA
 - Arrange
 - Set up the inputs and any dependencies the unit needs.
 - Act
 - Call the one method or operation under test
 - If you find yourself doing two things in this section, you're testing two things
 - Then you should probably split into two tests.
 - Assert
 - Verify the outcome.
 - One assertion per concept

```
@Test
void transferReducesSourceAccountBalance() {
    // Arrange
    Account from = new Account("ACC-001", new BigDecimal("100.00"));
    Account to = new Account("ACC-002", new BigDecimal("0.00"));

    // Act
    transferService.transfer(from, to, new BigDecimal("30.00"));

    // Assert
    assertThat(from.getBalance()).isEqualTo(new BigDecimal("70.00"));
}
```



Test Method

- Good test names
 - Describe behavior, not methods
 - Bad: TestTransfer1, testTransferMethod, testTransfer_2
 - Better: testTransferUpdatesBalances
 - Best: transferReducesSourceAccountBalance or transferRejectsAmountGreaterThanBalance
 - The test name should tell you what's being verified, without reading the body
 - When a test fails
 - The name appears in the report
 - A good name increases readability

```
@Test
void transferReducesSourceAccountBalance() {
    // Arrange
    Account from = new Account("ACC-001", new BigDecimal("100.00"));
    Account to = new Account("ACC-002", new BigDecimal("0.00"));

    // Act
    transferService.transfer(from, to, new BigDecimal("30.00"));

    // Assert
    assertThat(from.getBalance()).isEqualTo(new BigDecimal("70.00"));
}
```



Lifecycle Annotations

- Setup and teardown hooks
 - @BeforeAll and @AfterAll must be static
 - BeforeAll executed once before tests
 - Does general setup that applies to all tests
 - AfterAll executed once at the end of testing
 - Usually tears down what BeforeAll sets up
 - Most setup belongs in @BeforeEach, not @BeforeAll
 - BeforeEach runs before each test
 - Sets up the test baseline
 - For example, setting bank account balance to a specific value
 - AfterEach runs after each test
 - Undoes any side effects of running the test

```
class TransferServiceTest {  
  
    @BeforeEach  
    void setUp() {  
        // Runs before EVERY test method  
        transferService = new TransferService(...);  
    }  
  
    @AfterEach  
    void tearDown() {  
        // Runs after EVERY test method  
    }  
  
    @BeforeAll  
    static void initializeSuite() {  
        // Runs ONCE before all tests in this class  
    }  
  
    @AfterAll  
    static void cleanupSuite() {  
        // Runs ONCE after all tests in this class  
    }  
}
```



Assertions

- Common patterns
 - assertThrows
 - Tests that calling some code raises a specific exception
 - If the code does not throw, the test fails
 - If the code throws a different exception type, the test fails
 - If the code throws the expected exception, assertThrows returns it
 - assertAll
 - Runs multiple assertions and reports all failures at once
 - Without assertAll, the first failed assertion stops the test, and you don't see whether subsequent assertions would also have failed
 - With assertAll, you see all failures together
 - Use it for related assertions where you want a complete picture.

```
// Basic
assertEquals(expected, actual);
assertNotNull(value);
assertTrue(condition);

// Exception-throwing code
assertThrows(InsufficientFundsException.class, () -> transfer.execute());

// Multiple related assertions, all reported even if one fails
assertAll(
    () -> assertEquals(70, from.getBalance()),
    () -> assertEquals(30, to.getBalance()));

// Time-bounded operations
assertTimeout(Duration.ofMillis(500), () -> service.expensiveOperation())

// Confirmation that nothing was thrown
assertDoesNotThrow(() -> validator.validate(input));
```



Assertions

- Common patterns
 - `assertTimeout`
 - For testing operations that should complete within a bounded time
 - It does not abort the operation
 - If it takes too long; it just fails the test
 - Useful for catching performance regressions in critical paths
 - `assertDoesNotThrow`
 - For the rare case where the test's purpose is "confirming no exception is raised"
 - Most of the time, the absence of an exception is implicit
 - But sometimes you want to be explicit, especially when refactoring code that previously threw

```
// Basic
assertEquals(expected, actual);
assertNotNull(value);
assertTrue(condition);

// Exception-throwing code
assertThrows(InsufficientFundsException.class, () -> transfer.execute());

// Multiple related assertions, all reported even if one fails
assertAll(
    () -> assertEquals(70, from.getBalance()),
    () -> assertEquals(30, to.getBalance()));

// Time-bounded operations
assertTimeout(Duration.ofMillis(500), () -> service.expensiveOperation())

// Confirmation that nothing was thrown
assertDoesNotThrow(() -> validator.validate(input));
```



AssertJ

- Third-party assertion library
 - Recommend using over JUnit's built-in assertions
 - The main argument for AssertJ is readability
 - `assertThat(x).isEqualTo(y)`
 - Reads left-to-right naturally
 - `assertEquals(y, x)` has the expected value coming first, opposite to how you'd write it in natural language

```
// JUnit assertions
assertEquals("ACC-001", account.getAccountNumber());
assertTrue(account.getBalance().compareTo(BigDecimal.ZERO) > 0);

// AssertJ assertions
assertThat(account.getAccountNumber()).isEqualTo("ACC-001");
assertThat(account.getBalance()).isPositive();

// Chained assertions for richer checks
assertThat(account)
    .isNotNull()
    .extracting(Account::getStatus)
    .isEqualTo(AccountStatus.ACTIVE);

// Exception assertions read as English
assertThatThrownBy(() -> service.transfer(from, to, tooMuch))
    .isInstanceOf(InsufficientFundsException.class)
    .hasMessageContaining("ACC-001");
```



AssertJ

- Chained-assertion style on the slide
 - Productivity boost for tests that verify object state
 - Instead of writing five separate assertions, you chain them into one expression that reads like a description of the object
 - The exception assertion in the last example is genuinely better than JUnit's `assertThrows`
 - It assertively checks the type AND any properties of the exception in one expression

```
// JUnit: works but verbose
InsufficientFundsException ex = assertThrows(
    InsufficientFundsException.class,
    () -> service.transfer(from, to, tooMuch));
assertTrue(ex.getMessage().contains("ACC-001"));

// AssertJ: same intent, half the code
assertThatThrownBy(() -> service.transfer(from, to, tooMuch))
    .assertInstanceOf(InsufficientFundsException.class)
    .hasMessageContaining("ACC-001");
```



Parameterized Tests

- Parameterized tests
 - Run the same test method multiple times with different inputs
 - The example with blank account numbers is the canonical use case
 - The validator should reject blank input regardless of what kind of whitespace is in the string
 - Without parameterization you'd write five near-identical tests, one per whitespace variant
 - With `@ValueSource`, you write one test method and JUnit runs it five times, once per input

```
@ParameterizedTest
@ValueSource(strings = { "", " ", "   ", "\t", "\n" })
void rejectsBlankAccountNumber(String input) {
    assertThatThrownBy(() -> validator.validate(input))
        .isInstanceOf(ValidationException.class);
}

@ParameterizedTest
@CsvSource({
    "100.00, 30.00, 70.00",
    "100.00, 0.01, 99.99",
    "100.00, 100.00, 0.00"
})
void transferReducesSourceCorrectly(BigDecimal start, BigDecimal amount, BigDecimal expected) {
    Account from = new Account("ACC-001", start);
    transferService.transfer(from, accountB, amount);
    assertThat(from.getBalance()).isEqualTo(expected);
}
```



Parameterized Tests

- Parameterized tests
 - @ValueSource
 - Works for one parameter only
 - A list of strings, ints, longs, etc.
 - For multi-parameter tests, use @CsvSource
 - Each row of the CSV becomes one test invocation; the columns map to method parameters in order
 - @MethodSource
 - For cases where your test data needs to be constructed dynamically
 - Complex objects, calculated values, data loaded from a file
 - The annotation references a method that returns a Stream of arguments

```
@ParameterizedTest
@ValueSource(strings = { "", " ", " ", "\t", "\n" })
void rejectsBlankAccountNumber(String input) {
    assertThatThrownBy(() -> validator.validate(input))
        .isInstanceOf(ValidationException.class);
}

@ParameterizedTest
@CsvSource({
    "100.00, 30.00, 70.00",
    "100.00, 0.01, 99.99",
    "100.00, 100.00, 0.00"
})
void transferReducesSourceCorrectly(BigDecimal start, BigDecimal amount, BigDecimal expected) {
    Account from = new Account("ACC-001", start);
    transferService.transfer(from, accountB, amount);
    assertThat(from.getBalance()).isEqualTo(expected);
}
```



Example

- Shows
 - @DisplayName: human-readable class label
 - @BeforeEach for setup
 - @Test to mark the test method
 - @DisplayName on the method for a clear failure label
 - AAA structure with the body broken into Arrange / Act+Assert
 - Combined here, since assertThatThrownBy exercises both
 - AssertJ for the fluent assertion
 - Behavior-focused method name
 - transferRejectsAmountGreaterThanBalance

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .assertInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```



F.I.R.S.T.

- Properties of good rests
 - Fast
 - Thousands of tests should run in under a minute
 - Independent
 - Tests do not depend on order, on each other, or on shared mutable state
 - Repeatable
 - Same input, same result, every time, in any environment
 - Self-validating
 - Tests pass or fail; no human inspection required
 - Timely
 - Written close in time to the production code, not months later

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .isInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```



Style Guide

- Best practices
 - Behavior-focused names
 - AAA structure
 - AssertJ assertions
 - @Nested for organization
 - One logical assertion per test
 - Multiple physical assertions verifying the same outcome are fine
 - “Pass” condition may have multiple sub-conditions

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .isInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```



Style Guide

- Problematic
 - Test names that mirror method names
 - Testing implementation details
 - Tests with multiple Act steps
 - Should be split into multiple tests
 - Comments explaining what the test does
 - The test name should be self-explanatory
 - Thread.sleep(1000) to "wait for things"
 - Use proper synchronization or async testing instead

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .isInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```



Mocking with Mockito

- Problems with real dependencies
 - Slow
 - A database call adds milliseconds
 - Multiplied across hundreds of tests results in significant slowdowns
 - Non-deterministic
 - Network calls fail intermittently, time-dependent code returns different values
 - Expensive to set up
 - Spinning up a database or external service for every test is overkill for unit-level checks



Mocking with Mockito

- Problems with real dependencies
 - Out of your control
 - Third-party APIs may have rate limits, may be down, may charge per call
 - Hard to drive into edge cases
 - Making a real database simulate "connection dropped mid-transaction" is much harder than telling a mock to throw
 - Confounding errors
 - Problems in dependencies can give false positives for tests



Mocking with Mockito

- Terminology
 - Dummy
 - Placeholder object, never actually used; just satisfies a parameter
 - Stub
 - Returns canned answers to calls but has no logic, no verification
 - Fake
 - Has working implementation, but unsuitable for production like an in-memory DB
 - Mock
 - Pre-programmed with expectations
 - Verifies that calls happened correctly
 - Spy
 - Wraps a real object
 - You can override some methods, real ones still work



Mockito Basics

- Three basic operations
 - mock(T.class)
 - Creates an instance with all methods returning defaults
 - when(...).thenReturn(...)
 - Programs return values for specific calls
 - verify(...)
 - Asserts that specific calls did or did not happen

```
import static org.mockito.Mockito.*;

// 1. Create a mock
AccountRepository repo = mock(AccountRepository.class);

// 2. Stub its behavior
when(repo.findById("ACC-001")).thenReturn(Optional.of(account));

// 3. Verify how it was called (after exercising the unit under test)
verify(repo).findById("ACC-001");
verify(repo, times(1)).save(any(Account.class));
verify(repo, never()).delete(any(Account.class));
```



Mockito Annotations

- Cleaner way to wire mocks
 - `@ExtendWith(MockitoExtension.class)`
 - JUnit 5 extension that hooks Mockito into the test lifecycle
 - Without it, the `@Mock` and `@InjectMocks` annotations do nothing
 - Always include it
 - `@Mock`
 - Does what `mock(AccountRepository.class)` does, but on a field
 - Mockito creates a fresh mock before each test (so tests don't share state)
 - The reset-per-test behavior is exactly what you want
 - It's why we never need to write teardown code.

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @Mock
    private NotificationService notificationService;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferUpdatesAccount() {
        // accountRepository, notificationService are auto-created mocks
        // transferService is auto-constructed with those mocks injected
        ...
    }
}
```



Mockito Annotations

- **@InjectMocks**
 - Instructs Mockito to construct the annotated object using the test's other mocks for its dependencies
 - Mockito tries constructor injection first, then setter injection, then field injection
 - For a class like TransferService that takes its collaborators in the constructor, Mockito picks the constructor and supplies the mocks.
- A subtle point
 - @InjectMocks will not fail if some constructor parameters can't be matched to a mock, It just leaves them null
 - Occasionally a source of confusing test failures
 - You'll get a NullPointerException at runtime, not at test setup
 - The fix is to add the missing @Mock or to construct the unit under test manually in @BeforeEach

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @Mock
    private NotificationService notificationService;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferUpdatesAccount() {
        // accountRepository, notificationService are auto-created mocks
        // transferService is auto-constructed with those mocks injected
        ...
    }
}
```



Stubbing Methods

- `when()` / `thenReturn()` and variants
 - The multiple consecutive returns pattern
 - For testing code that calls the same method repeatedly and expects different results each time
 - A retry loop, for example: first call fails, second call succeeds.
 - The chained `thenReturn` calls let you express this.
- `thenThrow`
 - Essential for testing error paths
 - Use `thenThrow` to drive the unit under test into its error-handling code

```
// Single return value
when(repo.findById("ACC-001")).thenReturn(Optional.of(account));

// Different return values for consecutive calls
when(repo.findAll())
    .thenReturn(List.of(a1))      // first call
    .thenReturn(List.of(a1, a2)) // second call
    .thenReturn(List.of());      // third call onward

// Exception instead of a value
when(repo.findById("BAD-ID")).thenThrow(new DataAccessException("..."));

// Computed return value (useful for complex stubbing)
when(repo.save(any(Account.class))).thenAnswer(invocation -> {
    Account input = invocation.getArgument(0);
    return new Account(input.getAccountNumber(), input.getBalance());
});
```



Stubbing Methods

- `when()` / `thenReturn()` and variants
 - `thenReturnAnswer`
 - For cases where the return value depends on the input
 - Most of the time `thenReturn` is enough
 - Use `thenReturnAnswer` only when you need the mock to compute something based on what it received.

```
// Single return value
when(repo.findById("ACC-001")).thenReturn(Optional.of(account));

// Different return values for consecutive calls
when(repo.findAll())
    .thenReturn(List.of(a1))      // first call
    .thenReturn(List.of(a1, a2)) // second call
    .thenReturn(List.of());      // third call onward

// Exception instead of a value
when(repo.findById("BAD-ID")).thenThrow(new DataAccessException("..."));

// Computed return value (useful for complex stubbing)
when(repo.save(any(Account.class))).thenReturnAnswer(invocation -> {
    Account input = invocation.getArgument(0);
    return new Account(input.getAccountNumber(), input.getBalance());
});
```



Stubbing Methods

- Stubbing void methods
 - The `when().thenReturn()` syntax
 - Does not compile for void methods because there's no return type to specify
 - Mockito provides a different syntax for void methods
 - `do...().when()`, the inverse order
 - Most common case
 - `doThrow` for testing error paths
 - For example, a `NotificationService` that throws when SMTP is down lets you verify your code handles the exception correctly

```
// when().thenReturn() does NOT work for void methods (compiler error)
```

```
// Use doNothing() to make a void mock do nothing (this is actually  
// the default; explicit only for clarity)  
doNothing().when(notificationService).sendEmail(any(), any());
```

```
// Use doThrow() to make a void method throw  
doThrow(new NotificationException("smtp down"))  
    .when(notificationService).sendEmail(any(), any());
```

```
// Use doAnswer() for computed side effects  
doAnswer(invocation -> {  
    String to = invocation.getArgument(0);  
    sentEmails.add(to);  
    return null;  
}).when(notificationService).sendEmail(any(), any());
```



Stubbing Methods

- Stubbing void methods
 - doNothing is rarely needed
 - Because doing nothing is the default behavior of any mocked void method
 - Use it only when you've previously stubbed the method to throw and want to reset to no-op behavior in a specific test
 - The general pattern to remember
 - For non-void methods, when(...).thenReturn()
 - For void methods, doSomething().when(...)
 - The difference exists because of Java's syntax constraints, not because the concepts are different

```
// when().thenReturn() does NOT work for void methods (compiler error)
```

```
// Use doNothing() to make a void mock do nothing (this is actually  
// the default; explicit only for clarity)  
doNothing().when(notificationService).sendEmail(any(), any());
```

```
// Use doThrow() to make a void method throw  
doThrow(new NotificationException("smtp down"))  
    .when(notificationService).sendEmail(any(), any());
```

```
// Use doAnswer() for computed side effects  
doAnswer(invocation -> {  
    String to = invocation.getArgument(0);  
    sentEmails.add(to);  
    return null;  
}).when(notificationService).sendEmail(any(), any());
```



Argument Matchers

- Argument matchers
 - How you express "this call, with these arguments" precisely
- Three levels of specificity
 - Exact value ("ACC-001")
 - The call matches only when the argument equals "ACC-001".
 - Type matcher (anyString(), any(Account.class))
 - The call matches any argument of that type
 - Useful when you don't care about the specific value.

```
// Exact value
when(repo.findById("ACC-001")).thenReturn(account);
verify(repo).findById("ACC-001");

// "Anything of this type"
when(repo.findById(anyString())).thenReturn(account);
verify(repo).save(any(Account.class));

// Custom predicate
verify(repo).save(argThat(a -> a.getBalance().compareTo(BigDecimal.ZERO) > 0));

// Mixing matchers and literals: must use eq() for literals
verify(repo).updateBalance(eq("ACC-001"), any(BigDecimal.class));
// Wrong: verify(repo).updateBalance("ACC-001", any(BigDecimal.class));
```



Argument Matchers

- Three levels of specificity
 - Custom predicate (argThat(...))
 - The call matches when your lambda returns true. Use for complex assertions on arguments
 - You cannot mix literals and matchers in the same call
 - eq("ACC-001") is a matcher that matches when the argument equals "ACC-001".
 - Functionally identical to passing the literal, but it satisfies Mockito's all-or-nothing rule

// WRONG

```
verify(repo).updateBalance("ACC-001", any(BigDecimal.class));
```

// RIGHT

```
verify(repo).updateBalance(eq("ACC-001"), any(BigDecimal.class));
```



Verifying Interactions

- Asserting what was called
 - Verification is the "assert" half of mocking
 - Stubbing tells the mock how to behave
 - Verification asserts how the unit under test interacted with it.
 - The most common pattern
 - `verify(mock).method(args)`
 - Assert the call happened exactly once with the matching arguments
 - The default count of one is usually right for unit tests
 - A service method should call its repository the right number of times

```
// Called exactly once (the default)
verify(repo).findById("ACC-001");

// Called a specific number of times
verify(repo, times(2)).findById(any());
verify(repo, atLeast(1)).save(any());
verify(repo, atMost(3)).findById(any());

// Never called
verify(repo, never()).delete(any());

// No interactions at all with this mock
verifyNoInteractions(notificationService);

// Specific call order
InOrder order = inOrder(repo, notificationService);
order.verify(repo).save(any());
order.verify(notificationService).sendEmail(any(), any());
```



Verifying Interactions

- Asserting what was called
 - times(n), atLeast(n), atMost(n)
 - Gives specific count matching
 - Useful when the unit under test legitimately calls a method multiple times (a retry loop, a batch operation)
 - never()
 - Useful for negative assertions: "this code path should NOT trigger a notification"
 - It's how you express "the unit under test made the right decision NOT to do something"

```
// Called exactly once (the default)
verify(repo).findById("ACC-001");

// Called a specific number of times
verify(repo, times(2)).findById(any());
verify(repo, atLeast(1)).save(any());
verify(repo, atMost(3)).findById(any());

// Never called
verify(repo, never()).delete(any());

// No interactions at all with this mock
verifyNoInteractions(notificationService);

// Specific call order
InOrder order = inOrder(repo, notificationService);
order.verify(repo).save(any());
order.verify(notificationService).sendEmail(any(), any());
```



Verifying Interactions

- Asserting what was called
 - verifyNoInteractions
 - Form for "this mock should have been ignored entirely"
 - Useful when you want to verify that an unhappy path did not, for example, send any emails
 - inOrder
 - For cases where the order of calls matters
 - For example, "save the account first, then send the notification"
 - If the order is reversed, a failure during notification would leave inconsistent state
 - The inOrder verification asserts the sequence.

```
// Called exactly once (the default)
verify(repo).findById("ACC-001");

// Called a specific number of times
verify(repo, times(2)).findById(any());
verify(repo, atLeast(1)).save(any());
verify(repo, atMost(3)).findById(any());

// Never called
verify(repo, never()).delete(any());

// No interactions at all with this mock
verifyNoInteractions(notificationService);

// Specific call order
InOrder order = inOrder(repo, notificationService);
order.verify(repo).save(any());
order.verify(notificationService).sendEmail(any(), any());
```



Example

- Line by line
 - @ExtendWith(MockitoExtension.class) activates Mockito for the test class
 - @Mock creates the accountRepository mock automatically
 - @InjectMocks constructs transferService using the mock as its dependency
 - The Arrange section creates input data and stubs the repository's behavior
 - The Act section calls the unit under test
 - The Assert section verifies the repository was called with the correct updated state

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferCallsRepositoryWithUpdatedBalances() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));
        when(accountRepository.findById("ACC-001")).thenReturn(Optional.of(from));
        when(accountRepository.findById("ACC-002")).thenReturn(Optional.of(to));

        transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

        verify(accountRepository).save(argThat(a ->
            a.getAccountNumber().equals("ACC-001") &&
            a.getBalance().equals(new BigDecimal("70.00"))));
    }
}
```



Anti-Patterns

- Anti-Patterns to avoid
 - Over-mocking
 - Mocking five collaborators to test one method means the unit under test is doing too much
 - Mocking value objects
 - Never mock String, BigDecimal, LocalDate, your own DTOs; just construct them
 - Mocking what you don't own
 - Don't mock JDBC, Spring framework classes, JPA; use them or use Spring Boot Test
 - Brittle interaction tests
 - Verifying every method call is implementation-locked; verify outcomes instead
 - Thread.sleep and verify race conditions
 - If you need to wait for async behavior, use proper synchronization
 - Use the timeout and other features of Mockito

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferCallsRepositoryWithUpdatedBalances() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));
        when(accountRepository.findById("ACC-001")).thenReturn(Optional.of(from));
        when(accountRepository.findById("ACC-002")).thenReturn(Optional.of(to));

        transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

        verify(accountRepository).save(argThat(a ->
            a.getAccountNumber().equals("ACC-001") &&
            a.getBalance().equals(new BigDecimal("70.00"))));
    }
}
```



Anti-Patterns

- Anti-Patterns to avoid
 - Over-mocking
 - Mocking five collaborators to test one method means the unit under test is doing too much
 - Mocking value objects
 - Never mock String, BigDecimal, LocalDate, your own DTOs; just construct them
 - Mocking what you don't own
 - Don't mock JDBC, Spring framework classes, JPA; use them or use Spring Boot Test
 - Brittle interaction tests
 - Verifying every method call is implementation-locked; verify outcomes instead
 - Thread.sleep and verify race conditions
 - If you need to wait for async behavior, use proper synchronization
 - Use the timeout and other features of Mockito

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

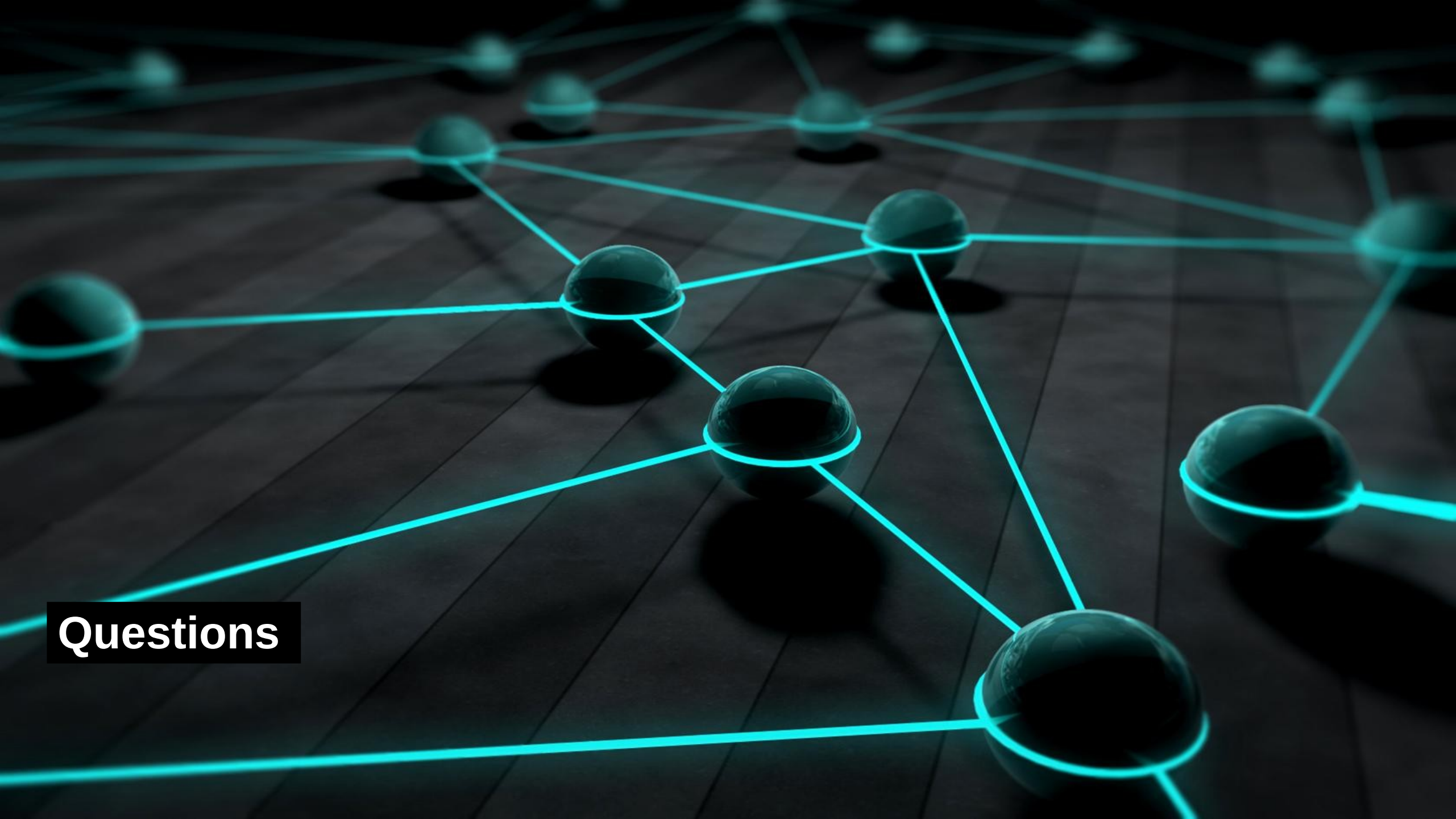
    @InjectMocks
    private TransferService transferService;

    @Test
    void transferCallsRepositoryWithUpdatedBalances() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));
        when(accountRepository.findById("ACC-001")).thenReturn(Optional.of(from));
        when(accountRepository.findById("ACC-002")).thenReturn(Optional.of(to));

        transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

        verify(accountRepository).save(argThat(a ->
            a.getAccountNumber().equals("ACC-001") &&
            a.getBalance().equals(new BigDecimal("70.00"))));
    }
}
```





Questions